

문자열의 알파벳 문자를 소문자로 바꾸어 준다	LOWER(문자열)
문자열의 알파벳 문자를 대문자로 바꾸어 준다	UPPER(문자열)
문자나 숫자를 ASCII 코드 번호로 바꾸어 준다	ASCII(문자)
ASCII 코드 번호를 문자나 숫자로 바꾸어 준다	CHR/CHR(ASCII번호)
문자열1과 문자열2를 연결한다	CONCAT(문자열1,문자열2)
문자열 m위치에서 n개에 해당하는 문자를 돌려준다	SUBSTR/SUBSTRING(문자열,m,n)
문자열의 개수를 숫자값으로 돌려준다	LENGTH/LEN(문자열)
문자열의 첫 문자부터 확인해서 지정문자가 나타나면 해당문자를 제거한다	LTRIM(문자열,지정문자)
문자열의 끝 문자부터 확인해서 지정문자가 나타나면 해당문자를 제거한다	RTRIM(문자열,지정문자)
문자열에서 머리말, 꼬리말, 양쪽에 있는 지정문자를 제거한다	TRIM([옵션] 지정문자 from 문자열)
숫자의 절대값을 돌려준다	ABS(숫자)
숫자가 양수인지, 음수인지, 0인지를 구별한다	SIGN(숫자)
숫자1을 숫자2로 나누어 나머지 값을 리턴한다	MOD(숫자1, 숫자2)
숫자보다 크거나 같은 최소 정수를 리턴한다 (올림)	CEIL/CEILING(숫자)
숫자보다 작거나 같은 최대 정수를 리턴한다 (내림)	FLOOR(숫자)
숫자를 소수점 m자리에서 반올림하여 리턴한다 (반올림)	ROUND(숫자, m)
숫자를 소수점 m자리에서 잘라서 버린다 (반내림)	TRUNC(숫자, m)
현재 날짜와 시각을 출력한다	SYSDATE/GETDATE()
날짜데이터에서 년/월/일 데이터를 뽑아낸다	EXTRACT/DATEPART(옵션 from 날짜)
문자열을 숫자로 변환한다	TO_NUMBER(문자열)
숫자나 날짜를 FORMAT 형태로 문자열 타입으로 변환한다	TO_CHAR(숫자 날짜,FORMAT)
문자열을 FORMAT 형태로 날짜 타입으로 변환한다	TO_DATE(문자열, FORMAT)
표현식1의 결과값이 NULL이면 표현식2의 값을 출력한다	NVL/ISNULL(표현식1,표현식2)
표현식1이 표현식2와 같으면 NULL을, 같지 않으면 표현식1을 리턴한다	NULLIF(표현식1,표현식2)
임의의 개수 표현식에서 NULL이 아닌 최초의 표현식을 나타낸다	COALESCE(표현식1,2,3,...)
NULL값을 포함한 행의 개수를 출력한다	COUNT(*)
표현식의 값이 NULL 값인 것을 제외한 행의 수를 출력한다	COUNT(표현식)
표현식의 NULL값을 제외한 합계를 출력한다	SUM(표현식)
표현식의 NULL값을 제외한 평균을 출력한다	AVG(표현식)
표현식의 최대값을 출력한다	MAX(표현식)
표현식의 최소값을 출력한다	MIN(표현식)
표현식의 표준편차를 출력한다	STDDEV(표현식)
표현식의 분산을 출력한다	VARIAN(표현식)
계층형질의] 루트데이터면1, 하위데이터면2. LEAF 데이터까지 1씩 증가	LEVEL
계층형질의] 해당데이터가 리프면1, 그렇지않으면 0	CONNECT_BY_ISLEAF
계층형질의] 해당데이터가 조상으로서 존재하면1, 그렇지않으면 0	CONNECT_BY_ISCYCLE
계층형질의] 루트데이터부터 현재전개할 데이터까지의 경로를 표시한다	SYS_CONNECT_BY_PATH(칼럼,분리자)
계층형질의] 현재전개할 데이터의 루트데이터를 표시한다	CONNECT_BY_ROOT 칼럼
소그룹 간의 소계를 계산. 병렬로 수행이 가능하며. 시간 및 데이터에 적합	ROLLUP
소규모 그룹 그룹 간 다차원적인 소계를 계산. ROLLUP에 비해 다양한 데이터	CUBE
원하는 항목에 대한 소계를 계산한다	GROUPING SETS
순위 구하기. 동일한 값은 동일한 순위 부여	RANK() over
순위 구하기. 동일 순위 시 하나의 건수로 취급	DENSE_RANK() over
순위 구하기. 동일한 순위는 없다	ROW_NUMBER() over

파티션 별로 윈도우의 합을 구한다	SUM(칼럼) over (partition
파티션 별로 윈도우의 최대값을 구한다	MAX(칼럼) over (partition
파티션 별로 윈도우의 최소값을 구한다	MIN(칼럼) over (partition
파티션 별로 윈도우의 평균을 구한다	AVG(칼럼) over (partition
윈도우의 정렬 결과 갯수를 센다	COUNT(*) over (order by
파티션 별 윈도우에서 가장 먼저 나온 값	FIRST_VALUE(칼럼) over (partition
파티션 별 윈도우에서 제일 마지막에 나온 값	LAST_VALUE(칼럼) over (partition
윈도우의 정렬 결과에서 이전 몇 번째 행의 값을 가져온다	LAG(인자1,2,3) over (order by
윈도우의 정렬 결과에서 이후 몇 번째 행의 값을 가져온다	LEAD(인자1,2,3) over (order by
파티션 별 윈도우의 전체 건수에서 현재 행보다 작거나 같은 건수에 대한 누적백분율. >0 & <= 1	CUME_DIST() over (partition
파티션 별 윈도우에서 제일 먼저 나오는 것은 0, 제일 늦게 나오는 것은 1로 하여 값이 아닌 행의 순서별 백분율. >=0 & <=1.	PERCENT_RANK() over (partition
윈도우의 정렬 결과에서 전체 건수를 인자값으로 N등분한 결과. 조 나누기	NTILE(숫자) over (order by
전체 SUM 값에 대한 행별 칼럼 값의 백분율을 소수점으로 나타낸다 >0 & <=1.	RATIO_TO_REPORT(칼럼)
현재 행을 기준으로 파티션 내의 첫 번째 행까지 범위를 지정 (반복 누적)	RANGE UNBOUNDED PRECEDING
현재 행을 기준으로 파티션 내의 앞의 한건, 뒤의 한건을 범위로 지정	ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
현재 행을 기준으로 앞에서 -50. 뒤 +150의 범위로 지정	RANGE BETWEEN 50 PRECEDING AND 150 FOLLOWING
현재 행을 포함해서 파티션 내의 마지막 행까지를 범위로 지정	ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
유저를 생성하고 권한을 제어하는데 대신 일을 해줄 중간역할을 하는 애	ROLE
이 역할을 하는 애의 2개의 주로 쓰는 권한 역할 (클래스 같은)	CONNECT/RESOURCE

PROCEDURE	USER DEFINED FUNCTION	TRIGGER
<pre> create OR REPLACE PROCEDURE p_DEPT_insert (v_DEPTNO in number, v_DNAME in varchar2, v_LOC in varchar2, v_result out varchar2) IS CNT number := 0; BEGIN SELECT COUNT(*) INTO CNT FROM DEPT WHERE DEPTNO = v_DEPTNO AND ROWNUM = 1; IF CNT > 0 THEN v_result := '이미 등록된 부서번호'; ELSE INSERT INTO DEPT (DEPTNO, DNAME, LOC) VALUES (v_DEPTNO, v_DNAME, v_LOC); COMMIT; v_RESULT := '입력 완료'; END IF; EXCEPTION WHEN OTHERS THEN ROLLBACK; v_RESULT := 'ERROR'; END; / </pre>	<pre> CREATE OR REPLACE FUNCTION UTIL_ABS (v_input in NUMBER) return NUMBER IS v_return number := 0; BEGIN IF v_input < 0 THEN v_return := v_input * -1; ELSE v_return := v_input; END IF; RETURN v_return; END; / </pre>	<pre> CREATE OR REPLACE TRIGGER SUMMARY_SALES AFTER INSERT ON ORDER_LIST FOR EACH ROW DECLARE o_date ORDER_LIST.ORDER_DATE%TYPE; o_prod ORDER_LIST.PRODUCT%TYPE; BEGIN o_date := :NEW.ORDER_DATE; o_prod := :NEW.PRODUCT; UPDATE SALES_PER_DATE SET QTY = QTY + :NEW.QTY, AMOUNT = AMOUNT + :NEW.AMOUNT WHERE SALE_DATE = o_date AND PRODUCT = o_prod; IF SQL%NOTFOUND THEN INSERT INTO SALES_PER_DATE VALUES(o_date, o_prod, :NEW.QTY, :NEW.AMOUNT); END IF; END; / </pre>

조인

NL 조인

- 랜덤 액세스 위주의 조인
- 한레코드씩 순차적으로 진행
- 테이블의 크기가 작을수록 좋음
- 온라인 프로그램에서 사용할 때 좋음
- Driving 테이블의 크기가 가장 많은 영향을 준다.

SMJ

- 조인하기전에 정렬
- 넓은 데이터 처리에 유리함
- 정렬에 필요한 데이터가 많아지면 메모리대신 디스크를 사용하므로 성능이 떨어질 수 있음
- 조인 조건의 인덱스 유무에 영향을 받지 않지만 Sort 가 필요하다.

Hash 조인

- 한쪽 테이블의 크기가 작을수록 좋음
- NL의 랜덤액세스, SMJ의 정렬작업에 대한 부담으로 등장
- 인덱스를 사용하지 않음, 인덱스 대신 Hash 함수를 사용
- 동등조인(=) 에서만 사용
- 수행 빈도가 높은 OLTP성 작업에서는 불리함
- 수행 빈도가 낮고, 쿼리 수행시간이 오래 걸리는, 대용량 테이블 조인 작업에 유리함(DW,OLAP성 쿼리)

1. CHAR VS VARCHAR

CHAR에서는 문자열을 비교할 때 공백(BLANK)을 채워서 비교하는 방법을 사용한다. 공백 채우기 비교에서는 우선 짧은 쪽의 끝에 공백을 추가하여 2개의 데이터가 같은 길이가 되도록 한다. 그리고 앞에서부터 한 문자씩 비교한다. 그렇기 때문에 끝의 공백만 다른 문자열은 같다고 판단한다. 그에 반해 VARCHAR 유형에서는 맨 처음부터 한 문자씩 비교하고 공백도 하나의 문자로 취급하므로 끝의 공백이 다르면 다른 문자로 판단한다.

ex) CHAR 유형 'AA' = 'AA '
 VARCHAR 유형 'AA' <> 'AA '

2. CHECK 제약조건

CHECK(COL1 IN (1,2,3))

CHECK 제약조건이 걸린 컬럼이 NOT NULL인 경우 NULL 값이 들어갈 수 없지만 NOT NULL이 아니면 NULL 값 허용

3. SELECT 문장을 통한 테이블 생성 사례

1) ORACLE

CTAS : CREATE TABLE ~ AS SELECT ~

2) SQL SERVER

SELECT * INTO TABLE1FROM TABLE2

※ 기존 테이블의 제약조건 중에 NOT NULL만 새로운 복제 테이블에 적용되고, 기본키, 고유키, 외래키 , CHECK 등의 다른 제약조건은 없어진다.

4. RENAME COLUMN

1) ORACLE

ALTER TABLE XPDLMFAUD

RENAME COLUMN 변경해야 할 컬럼명 TO 새로운 컬럼명;

2) SQL SERVER

SP_RENAME 변경해야할 컬럼명, 새로운 컬럼명, 'COLUMN';

5. RENAME TABLE

1) ORACLE

RENAME 변경 전 테이블 명 TO 변경 후 테이블명;

2) SQL SERVER

SP_RENAME 변경 전 테이블명, 변경 후 테이블명;

6. 연산자의 종류

1) 비교 연산자 : =, >, >=, <, <=

2) SQL 연산자 : BETWEEN A AND B, IN(LIST), LIKE '비교문자열', IS NULL

3) 논리 연산자 : AND, OR, NOT

7. 연산자의 우선순위

괄호() -> NOT 연산자 -> 비교 연산자, SQL 비교연산자 -> AND -> OR

8. ROWNUM

WHERE 절에서 행의 개수를 제한하는 목적으로 사용

한 행만 가져오고 싶을 때 WHERE ROWNUM = 1;

WHERE ROWNUM <= 1;

WHERE ROWNUM < 2;

두건 이상의 N행을 가져오고 싶을 때 WHERE ROWNUM = N; (X)

WHERE ROWNUM <= N; (O)

WHERE ROWNUM < N+1; (O)

9. 단일행 함수의 종류

1) 문자형 함수 : LOWER, UPPER, SUBSTR / SUBSTRING, LENGTH / LEN, LTRIM, RTRIM, TRIM, ASCII

2) 숫자형 함수 : ABS, MOD, ROUND, TRUNC, SIGN, CHR / CHAR, CEIL / CEILING, FLOOR, EXP, LOG, LN, POWER, SIN, COS, TAN

3) NULL 관련 함수 : NVL / ISNULL, NULLIF, COALESCE

※ IS NULL은 함수가 아니라 연산자이다.

10. CEIL / CEILING() VS FLOOR()

1) CEIL / CEILING() : 숫자보다 크거나 같은 최소 정수를 리턴한다.

ex) CEIL(38.123) / CEILING(38.123) -> 39

CEILING(-38.123) -> 38

2) FLOOR() : 숫자보다 작거나 같은 최대 정수를 리턴한다.

ex) FLOOR(38.123) -> 38

FLOOR(-38.123) -> -39

※ CEILING은 천장, FLOOR은 바닥을 연상해서 외우면 좋을 듯 싶다.

11. NVL / ISNULL 함수를 이용해 공집합을 9999로 바꾸기

SELECT NVL(MGR, 9999) MGR FROM EMP WHERE ENAME = 'JSC';

데이터를 찾을 수 없다.

-> NVL / ISNULL 함수는 NULL 값을 대상으로 다른 값으로 바꾸는 함수이지 공집합을 대상으로 하지 않는다.

SELECT NVL(MAX(MGR), 9999) MGR FROM EMP WHERE ENAME = 'JSC';

-> 집계함수를 인수로 한 NVL / ISNULL 함수를 이용해서 공집합인 경우에도 빈칸이 아닌 999로 출력하게 한다. 다른 함수와 달리 집계함수나 SCALAR SUBQUERY의 경우는 인수의 결과 값이 공집합인 경우에도 NULL을 출력한다.

12. NULLIF 함수

NULLIF(EXPR1, EXPR2)

EXPR1이 EXPR2와 같으면 NULL, 다르면 EXPR1을 리턴

13. COALESCE 함수

COALESCE(EXPR1, EXPR2, ...)

임의의 개수 EXPR에서 NULL이 아닌 최초의 EXPR을 나타낸다.

만약 모든 EXPR이 NULL이면 NULL을 리턴

14. 집계함수

COUNT(*) : NULL 값을 포함한 행의 수

COUNT(표현식) : 표현식의 값이 NULL 값인 것을 제외한 행의 수

SUM() : NULL 값을 제외한 합계

AVG() : NULL 값을 제외한 평균

※ 조건절에 해당하는 데이터가 없을 때 COUNT(*)의 결과 값은 0

COL1	COL2	COL3	COL4
NULL	NULL	50	30
30	20	10	30
NULL	10	NULL	NULL

SUM(COL1) + SUM(COL2 + COL3) + SUM(COL4)의 값은?

1.120 2.180 3.NULL 4.90

답은 120

15. GROUP BY 절

1) GROUP BY 절에서는 ALIAS명을 사용할 수 없다.

2) WHERE 절은 전체 데이터를 GROUP으로 나누기 전에 행들을 미리 제거시킨다.

3) HAVING 절은 일반적으로 GROUP BY 절 뒤에 위치한다.

4) GROUP BY, HAVING 절에는 SELECT 절에 정의되지 않은 컬럼은 사용 못함

※ ORDER BY 절을 SELECT 절에 정의되지 않은 컬럼 사용 가능

16. NULL의크기

ORACLE에서는 NULL 값을 가장 큰 값으로 간주

SQL SERVER에서는 NULL 값을 가장 작은 값으로 간주

17. NATURAL JOIN

1) 두 테이블 간의 동일한 이름을 갖는 모든 칼럼들에 대해 EQUI(=) JOIN을 수행

2) NATURAL JOIN이 명시되면 추가로 USING 조건절, ON 조건절, WHERE 절에서 JOIN 조건을 정의할 수 없다.

3) JOIN에 사용된 컬럼들은 같은 데이터 유형이어야 하며, ALIAS나 테이블명과 같은 접두사를 붙일 수 없다.

18. USING 조건절

FROM 절에 USING 조건절을 이용하면 같은 이름을 가진 컬럼들 중에서 원하는 컬럼에 대해서만 선택적으로 EQUI JOIN을 할 수 있다.

SELECT * FROM DEPT JOIN DEPT_TEMP

USING (DEPTNO);

※ JOIN 칼럼에 대해서는 ALIAS나 테이블 이름과 같은 접두사를 붙일 수 없다.

19. UNION VS UNION ALL

- 1) UNION : 여러개의 SQL의 결과에 대한 합집합으로 결과에서 모든 중복된 행을 하나의 행으로 만든다.
- 2) UNION ALL : 중복된 행도 그대로 결과에 표시한다.

20. 서브쿼리

- 1) 서브쿼리란 하나의 SQL문안에 포함되어 있는 또 다른 SQL문을 말한다.
- 2) 서브쿼리는 메인쿼리의 칼럼을 모두 사용할 수 있지만 메인쿼리는 서브쿼리의 칼럼을 사용할 수 없다. 질의 결과에 서브쿼리의 칼럼을 표시해야 한다면 조인방식으로 변환하거나 함수, 스칼라 서브쿼리 등을 사용해야 한다.
- 3) 단일행 서브쿼리
서브쿼리가 단일행 비교 연산자 (=, <, <=, >, >=, <>)와 함께 사용할 때는 서브쿼리의 결과건수가 반드시 1건 이하이어야 한다.
- 4) 다중행 서브쿼리
서브쿼리의 결과가 2건 이상 반환될 수 있다면 반드시 다중행 비교연산자 (IN, ALL, ANY, SOME)와 함께 사용해야 한다.
ex) IN (서브쿼리), ALL(서브쿼리), ANY(서브쿼리), EXISTS(서브쿼리)
- 5) 서브쿼리는 서브쿼리 레벨과 상관없이 항상 메인쿼리 레벨로 결과집합이 생성된다.
예를들어, 메인쿼리로 조직(1), 서브쿼리로 사원(M) 테이블을 사용하면 결과집합은 조직(1) 레벨이 된다.
- 6) 서브쿼리에서는 ORDER BY 절을 사용하지 못한다. ORDER BY 절은 SELECT 절에서 오직 한개만을 수 있기 때문에 ORDER BY 절은 메인쿼리의 마지막 문장에 위치해야 한다.

21. ORACLE의 INDEX 액세스 기법의 종류

INDEX UNIQUE SCAN, INDEX RANGE SCAN, INDEX RANGE SCAN DESCENDING,
INDEX FULL SCAN, FAST FULL INDEX SCAN, INDEX SKIP SCAN

22. 계층형 질의

- 1) START WITH 절은 계층구조 전개이 시작 위치를 지정하는 구문이다.
- 2) CONNECT BY 절은 다음에 전개될 자식 데이터를 지정하는 구문이다.
- 3) 루트 데이터는 LEVEL 1이다. (0이 아님)